

1.
 - a. Static scoping determines a variables value based on where the code was written. If it isn't found in the current scope, then it will check through the parent scopes. Dynamic scoping determines a variables value based on the call stack at run time. If the variable isn't found in the current function, then the program will search through callers starting with the most recent one.
 - b. Python uses static scoping while Bash uses dynamic scoping.
 - c. With dynamic scoping, a free variable can refer to different declarations depending on which functions were called, which means that a programmer would need to know the call history before knowing the value of the variable. On the other hand, with static scoping, a programmer can determine what a free variable is by reading the source code and looking at the scopes surrounding the function.
2. Lexical nesting (static): B -> Global (x = 10); prints x = 10
Call stack (dynamic): B -> A (x = 1) -> Global (x = 10); prints x = 1
3. Scope hole happens when a variable declared inside a function shadows another declaration. While an outer variable can still exist, it cannot be accessed within the inner function because the name shadows the outer functions variable name.

```
outer():  
    x = 10
```

```
inner(x)  
    print(x)  
    x += 1  
    print(x)
```

4. Resolving a free variable dynamically is dangerous in concurrent or multi-threaded interpreters because it uses the call stack to find variables. This means that, if one thread were to remove or add a function call while another thread is searching the call stack for a variable, then the program could find the wrong variable.